1. **Scenario Name:** ***Search Functionality***

Weak Prompt: "Test the search feature."

- **What I Feel is Missing**

1. The prompt misses the context, Instructions to return the more appropriate results.
2. it is a very generic prompt
3. It does not specify the user application or any other examples required by the chat Agent.

- **Weak Prompt Response (Chatagent: Gemini)**

I'm ready! If you have something specific, you'd like to look up, just let me know what you're curious about. We can check out recent news, dive into a complex topic, or find specific information you need.

What are we searching for first?

- **Rewritten Prompt**

C – Context

As a Manual quality assurance, I would need to build Step by step test case document for the search functionality of an ecommerce application.

I – Instruction

[Mandatory] The test case document should include 10 scenarios and step by step process for each of the scenarios

[Strict] The test case document should include both negative, positive scenarios and boundary value conditions be mentioned in the test document

[Mandatory] The document should include the environment it is to be tested.AS I would be reusing the document for testing in QA,UAT,Production environments. I would need to categorize the cases in a way that they can be reused with out having it rewrite again.

[Critical] Add cases to verify that the search functionality should include a filter to get a more refined search

[Mandatory] Add cases to verify the search option provides Autocomplete & Predictive Suggestions

[Mandatory] Add cases to verify the search option autocorrects spelling mistakes when user types

[Low Priority] Include the testing type if it can be specific to smoke, sanity or regression testing.

E- Example:

Test scenario ID| Test scenario Description |Test Case ID| Test case description | Priority | Testing Type | Precondition | Test data Expected Result| Actual Result

P - Persona

As a Performance test engineer please provide me load testing suggestion for the search functionality.

Also, as a Test lead I would need to make process improvements, so also suggest me a more industry standard documents to be used by my organisation.

O - Output Format

Please provide the test case document in a excel format

T – Tone

The test case document needs to be in professional tone

- **ICEPOT Prompt Response:**

Executive Overview of the Master Test Suite

The generated spreadsheet, Search_Functionality_Master_Test_Suite.xlsx, has been fully built using professional, enterprise-grade engineering practices. It utilizes a unified visual language with a **Classic Navy palette** (#1B365D), subtle zebra-striping, explicit multi-line cell formatting, auto-fitted margins, and structural freeze-panes on key tabular data matrices for high usability.

The workbook is split into three logically decoupled tabs to facilitate reusability across **QA, UAT, and Production** tracking cycles without rewriting any steps. Below is a breakdown of the design architecture and structural strategies included inside the workbook:

Tab 1: Metadata & Environment Matrix

To prevent test script fragmentation across different releases and target clouds, a **Dynamic Tokenization System** is utilized.

- Test steps in Tab 2 reference variables such as {{BASE_URL}}, {{VALID_TERM}}, {{MISSPELLED}}, and {{FILTER_CAT}}.
- Tab 1 contains a matrix mapping these tokens directly to their respective environment values (e.g., seeding synthetic items for QA, utilizando staging values for UAT, and leveraging real, high-availability public stock queries for the Production environment).
- To execute a testing cycle on a specific tier, simply swap out the context inputs from this centralized token directory.

Tab 2: Test Case Directory (10 Scenarios)

This tab features a fully formatted matrix mapped exactly to your requested layout parameters. It blends **Positive**, **Negative**, **Boundary Value Analysis (BVA)**, and **UI/UX Stability** scenarios, explicitly assigning **Testing Types** (*Smoke, Sanity, and Regression*) to facilitate smart test execution scoping.

Here is an overview of the 10 exhaustive scenarios compiled inside the sheet:

1. **TS_SRCH_01 (Positive | Sanity/Regression):** Validation of *Autocomplete & Predictive Suggestions* overlay rendering on sequential character entry, evaluating prefix highlighting.
2. **TS_SRCH_02 (Positive | Critical | Regression):** Evaluation of *Search Multi-Category Filter Panels* to ensure accurate results grid refinement without a full page reload breaking state.
3. **TS_SRCH_03 (Positive | High | Sanity/Regression):** Validation of *Spelling Autocorrection* algorithms and fallback message prompts ("Showing results for [Corrected Term]. Did you mean...").
4. **TS_SRCH_04 (Positive | High | Smoke/Sanity):** Standard exact alphanumeric match logic ensuring seamless direct navigation or high relevancy scores.
5. **TS_SRCH_05 (Boundary | Medium | Regression):** Input constraint verification matching extreme boundaries (Minimum: 1 Character threshold vs Maximum: 128 Characters truncation limits).
6. **TS_SRCH_06 (Negative | High | Regression):** Security and code injection resilience testing, passing raw SQL strings (' OR 1=1 --) and Cross-Site Scripting (<script>) elements to verify proper backend string sanitization and 500 error prevention.
7. **TS_SRCH_07 (Negative | Medium | Sanity/Regression):** Whitespace and null-string validation, assessing the system's string-trimming routines to prevent submission of blank queries.
8. **TS_SRCH_08 (Negative | Medium | Sanity):** Validation of the gracefully managed "0 Results Found / No Results Grid" fallback experience, ensuring alternative suggestions or popular tags are surface-rendered.
9. **TS_SRCH_09 (Positive | High | Regression):** Multi-facet filtering combination loops, verifying that single filter pill removal ("X" action) and master "Clear All" actions correctly recalculate product scopes.
10. **TS_SRCH_10 (UI Stability / Boundary | Low | Regression):** Async UI stability evaluation evaluating input debouncing delay (~300ms) under highly rapid typing and backspacing to prevent front-end race conditions.

Tab 3: Performance & Test Lead Strategy

1. Performance Load Testing Architecture

As a Performance Engineer, search functionality requires deep scrutiny because text parsing, query matching, and multi-facet indexing can heavily strain database and caching tiers. The strategy implemented in the sheet outlines:

- **Load Profile Modeling:** Designing a profile combining concurrent Peak Load (e.g., 500 concurrent users) and sudden Spike Loads (e.g., 2000 concurrent users). It maps a **70/20/10 operational split** (70% simple keywords, 20% heavy filter sorting, 10% high-frequency autocomplete network requests).
- **Target SLAs / KPIs:** Establishing mandatory non-functional thresholds—such as ensuring that Autocomplete API response bounds remain strictly < 200ms for seamless typing, search landing grid requests render in < 1.5s, and error counts stay under < 0.5%.
- **Caching & Tool Layering:** Recommending performance engines like **Apache JMeter** or **Gatling** parameterized with extensive dynamic keyword datasets to intentionally bypass CDN edge caching layers, ensuring database optimization, ElasticSearch index health, and cache hit ratios can be audited accurately.

2. Test Lead Process Improvements & Quality Framework Documents

To elevate an organization's maturity level, relying solely on test cases creates siloed execution. Tab 3 introduces standard blueprints for four foundational quality governance documents to establish standard processes:

- **Master Test Strategy (MTS):** Standardizes cross-team alignment, tool configuration, environment criteria, and environment hand-off parameters.
- **Requirement Traceability Matrix (RTM):** Links business requirements directly to technical design criteria and specific test case IDs, ensuring 100% test coverage visibility and removing execution gaps.
- **Automation Framework Architecture Guide:** Establishes coding standards, standard locator patterns (e.g., Page Object Model), and CI/CD pipelines (Jenkins/GitHub Actions) to maximize automation reusability.
- **Defect Lifecycle & Triage Charter:** Outlines clear defect logging standards, classification definitions (Severity vs. Priority), and automated escalation paths to streamline communication between engineering and QA squads.

Technical File Verification

The file contains optimized structural parameters including:

- Wrap-text configurations for dense text boxes (such as steps and expected outcomes).
- Hard column widths ranging from 12 to 45 units to eliminate clipping.
- openpyxl.freeze_panes = "A2" on the Test Cases tab to keep column headers locked during rapid scrolling.

Search_Functionality_
Master_Test_Suite.xls›

- **Observations**

1. When the weak prompt was given the Chat agent could not proceed as the prompt was lacking details. The same prompt when written with ICEPOT framework chat agent had given the desired output
2. Step by step instructions and labelling them help to provide more accurate results
3. Additional persona improved the results by providing more details on performance testing and process improvements.

Scenario 2: **Shopping Cart**

Weak Prompt: "Test the cart functionality."

- **What I Feel is Missing**

1. The prompt Is missing the Application Context

2. The requirements related to the cart with respect to the application.
3. The scope and type are testing.

- **Weak Prompt Response (Chatagent: ChatGpt (OpenAI))**

Add to Cart

- Add a single item to the cart.
- Add multiple different items.
- Add the same item multiple times.
- Add items with different variants (size, color, etc.).
- Verify cart count updates correctly.

Update Cart

- Increase item quantity.
- Decrease item quantity.
- Set quantity to maximum allowed value.
- Attempt invalid quantities (0, negative, non-numeric).
- Verify subtotal and total recalculate correctly.

Remove Items

- Remove a single item.
- Remove all items.
- Verify cart becomes empty and displays the correct message.

Pricing & Discounts

- Verify item prices match product pages.
- Apply valid coupon codes.
- Test invalid/expired coupons.
- Verify taxes, shipping, and discounts are calculated correctly.
- Confirm totals update after quantity changes.

Persistence

- Refresh the page and verify cart contents remain.
- Log out and log back in (if applicable).
- Close and reopen the browser.

- Test cart persistence across devices for logged-in users.

Inventory Validation

- Add an in-stock item.
- Attempt to exceed available stock.
- Verify out-of-stock items are handled correctly.

Checkout Integration

- Proceed to checkout from the cart.
- Verify cart contents transfer correctly.
- Navigate back and ensure cart remains unchanged.

Edge Cases

- Add and remove items rapidly.
- Test with slow network conditions.
- Open the cart in multiple tabs and verify synchronization.
- Test guest vs. authenticated users.

Mobile & Browser Testing

- Verify cart functionality on mobile, tablet, and desktop.
- Test across major browsers (Chrome, Safari, Firefox, Edge).

Expected Result

At every step:

- Cart contents are accurate.
- Prices and totals are correct.
- No errors or data loss occur.
- UI updates immediately and consistently.

If you're testing a specific website or application, I can create detailed test cases with steps, expected results, and priority levels.

- **Rewritten Prompt**

C – Context

I am a Senior Software Test Engineer working on an Automobile Insurance platform where dealership users purchase insurance contracts for customers.

The Cart module allows dealers to:

- Add insurance products to cart
- View product premium
- Update vehicle information
- Remove products
- Purchase contracts
- Checkout

I – Instructions

[Strict] Generate comprehensive test cases covering:

- Positive scenarios
- Negative scenarios
- Boundary Value Analysis
- Edge Cases

[Mandatory] Cover validations for:

- VIN
- Odometer
- Vehicle Purchase Date
- In-Service Date
- Vehicle Condition
- Finance Type
- Contract Term

[Mandatory] VIN must be validated as:

- Required field
- Exactly 17 characters
- Alphanumeric
- Invalid lengths rejected

[Critical]

Validate dealer product eligibility:

- Dealer should only view products mapped to that dealer

- Unauthorized products must not be displayed

- API and DB validation required

[Mandatory]

Include:

- UI Test Cases

- API Test Cases

- Database Test Cases

- Security Test Cases

[Automation]

For every UI test case provide:

- Playwright TypeScript script

- Locator strategy

- Assertions

E – Output Format

Test Scenario ID |

Test Scenario Description |

Test Case ID |

Test Case Description |

Priority |

Testing Type |

Precondition |

Test Data |

Expected Result |

Actual Result |

Status

P – Persona

Act as a Senior QA Architect and Automation Engineer with expertise in Automobile Insurance applications and Playwright automation.

T – Tone

Professional, enterprise-grade, audit-ready test documentation suitable for QA, UAT, and regulatory review.

- ICEPOT Prompt Response:
- This is a very large scope (UI + API + DB + Security + Playwright for every test case). In a real-world QA document, this would easily exceed 200+ test cases.
- Below is a representative enterprise-grade test suite covering the highest-risk scenarios for the Cart module.
- ---
- Test Scenario: Cart – Add Insurance Product

Test Scenario ID	Test Scenario Description	Test Case ID	Test Case Description	Priority	Testing Type	Precondition	Test Data	Expected Result	Actual Result	Status
CART-001	Add Product to Cart	TC-001	Verify dealer can add eligible insurance product to cart	P0	Smoke/UI	Dealer logged in	Eligible Product	Product added successfully	TBD	Not Executed
CART-001	Add Product to Cart	TC-002	Verify dealer cannot add unauthorized product	P0	Regression/UI	Dealer logged in	Unauthorized Product	Product not displayed/blocked	TBD	Not Executed
CART-001	Add Product to Cart	TC-003	Verify multiple eligible products can be added	P1	Regression/UI	Dealer logged in	Multiple Products	Products added successfully	TBD	Not Executed
CART-001	Add Product to Cart	TC-004	Verify cart premium recalculates after	P0	Sanity/UI	Product Available	Insurance Product	Updated premium displayed	TBD	Not Executed

Test Scenario ID	Test Scenario Description	Test Case ID	Test Case Description	Priority	Testing Type	Precondition	Test Data	Expected Result	Actual Result	Status
			adding product							

- ---
- Test Scenario: VIN Validation

Test Scenario ID	Test Scenario Description	Test Case ID	Test Case Description	Priority	Testing Type	Precondition	Test Data	Expected Result	Actual Result	Status
VIN-001	VIN Validation	TC-005	Verify VIN is mandatory	P0	Smoke/UI	Cart Page Open	Blank VIN	Error displayed	TBD	Not Executed
VIN-001	VIN Validation	TC-006	Verify VIN accepts exactly 17 characters	P0	Smoke/UI	Cart Page Open	1HGCM82633A123456	Accepted	TBD	Not Executed
VIN-001	VIN Validation	TC-007	Verify VIN with 16 characters	P0	Regression/UI	Cart Page Open	1HGCM82633A12345	Validation Error	TBD	Not Executed
VIN-001	VIN Validation	TC-008	Verify VIN with 18 characters	P0	Regression/UI	Cart Page Open	1HGCM82633A1234567	Validation Error	TBD	Not Executed
VIN-001	VIN Validation	TC-009	Verify VIN accepts alphanumeric values	P0	Regression/UI	Cart Page Open	1HGCM82633A123456	Accepted	TBD	Not Executed
VIN-001	VIN Validation	TC-010	Verify VIN rejects special characters	P0	Regression/UI	Cart Page Open	1HGCM@2633A123456	Validation Error	TBD	Not Executed

- Boundary Values

Test Case ID	Input Length	Expected Result
TC-011	16	Reject
TC-012	17	Accept
TC-013	18	Reject

- Test Scenario: Odometer Validation

Test Case ID	Test Case Description	Priority	Expected Result
TC-014	Verify odometer is mandatory	P0	Error displayed
TC-015	Verify odometer accepts minimum value	P0	Accepted
TC-016	Verify odometer accepts maximum supported value	P0	Accepted
TC-017	Verify negative odometer value	P0	Rejected
TC-018	Verify alphabetic characters	P1	Rejected
TC-019	Verify decimal value	P1	Based on business rule

- Test Scenario: Vehicle Purchase Date

Test Case ID	Test Case Description	Priority	Expected Result
TC-020	Verify purchase date mandatory	P0	Validation message
TC-021	Verify past date accepted	P0	Accepted
TC-022	Verify current date accepted	P0	Accepted
TC-023	Verify future date rejected	P0	Validation Error

- Test Scenario: In-Service Date

Test Case ID	Test Case Description	Priority	Expected Result
TC-024	Verify in-service date mandatory	P0	Validation Error
TC-025	Verify in-service date before purchase date	P0	Validation Error
TC-026	Verify in-service date equal to purchase date	P0	Accepted
TC-027	Verify future in-service date	P1	Based on Business Rule

---

- Test Scenario: Vehicle Condition

Test Case ID	Test Case Description	Priority	Expected Result
TC-028	Verify vehicle condition mandatory	P0	Validation Error

Test Case ID	Test Case Description	Priority	Expected Result
TC-029	Verify New vehicle selection	P0	Accepted
TC-030	Verify Used vehicle selection	P0	Accepted
TC-031	Verify invalid value via API tampering	P0	Rejected

- Test Scenario: Finance Type

Test Case ID	Test Case Description	Priority	Expected Result
TC-032	Verify finance type mandatory	P0	Validation Error
TC-033	Verify Loan option	P0	Accepted
TC-034	Verify Lease option	P0	Accepted
TC-035	Verify Cash option	P0	Accepted
TC-036	Verify invalid finance type submitted	P0	Rejected

- Test Scenario: Contract Term

Test Case ID	Test Case Description	Priority	Expected Result
TC-037	Verify contract term mandatory	P0	Validation Error
TC-038	Verify minimum term	P0	Accepted
TC-039	Verify maximum term	P0	Accepted
TC-040	Verify unsupported term value	P0	Rejected

- API Validation Test Cases

Test Case ID	Description	Priority
API-001	Verify Add-to-Cart API returns 200	P0
API-002	Verify invalid VIN returns 400	P0
API-003	Verify unauthorized dealer returns 403	P0
API-004	Verify invalid token returns 401	P0
API-005	Verify premium calculation API accuracy	P0
API-006	Verify contract creation API response	P0
API-007	Verify cart deletion API	P1
API-008	Verify API schema validation	P1
API-009	Verify response time under SLA	P1

- Database Validation Test Cases

Test Case ID	Description	Priority
DB-001	Verify cart record created after add-to-cart	P0
DB-002	Verify VIN stored correctly	P0
DB-003	Verify premium matches UI value	P0
DB-004	Verify contract term stored correctly	P0
DB-005	Verify dealer-product mapping enforced	P0

Test Case ID	Description	Priority
DB-006	Verify deleted cart removed from DB	P1
DB-007	Verify audit trail created	P1

- Sample Query
- SELECT *
  FROM CART
  WHERE VIN='1HGCM82633A123456';
- ---
- Security Test Cases

Test Case ID	Description	Priority
SEC-001	Verify authentication required for cart access	P0
SEC-002	Verify authorization for dealer-specific products	P0
SEC-003	Verify SQL Injection in VIN field	P0
SEC-004	Verify XSS attack in VIN field	P0
SEC-005	Verify CSRF protection	P0
SEC-006	Verify session timeout handling	P0
SEC-007	Verify JWT token tampering	P0
SEC-008	Verify API parameter tampering	P0
SEC-009	Verify dealer cannot access another dealer's cart	P0
SEC-010	Verify dealer data isolation	P0

- ---
- Playwright Automation Example (VIN Mandatory Validation)
- Locator Strategy
- vinInput = page.locator('#vin');
  continueBtn = page.locator('#continue');
  errorMessage = page.locator('.validation-error');
- Playwright Script
- import { test, expect } from '@playwright/test';

  test('VIN Mandatory Validation', async ({ page }) => {

    await page.goto('/cart');

    await page.locator('#vin').fill('');

    await page.locator('#continue').click();

    await expect(
      page.locator('.validation-error')
).toContainText('VIN is required');

  });
- ---
- Playwright Automation Example (17 Character VIN Validation)

- import { test, expect } from '@playwright/test';

  test('Validate 17 Character VIN', async ({ page }) => {

    await page.goto('/cart');

    await page.locator('#vin')
      .fill('1HGCM82633A123456');

    await page.locator('#continue').click();

    await expect(
      page.locator('.validation-error')
).toHaveCount(0);

  });
- ---
- Playwright Automation Example (Dealer Product Eligibility)
- import { test, expect } from '@playwright/test';

  test('Dealer should view only eligible products', async ({ page }) => {

    await page.goto('/products');

    const visibleProducts =
      await page.locator('.product-card').allTextContents();

    expect(visibleProducts).toContain('Gold Protection');

    expect(visibleProducts).not.toContain('Restricted Product');

  });
- Recommended Coverage Summary

Area	Count
UI Functional	40+
API	20+
Database	15+
Security	20+
Boundary Tests	15+
Automation Candidates	80%+

- For a production automobile insurance platform, I would typically expand this into 100–150 detailed test cases with complete Playwright Page Object Model (POM), API automation, SQL validations, traceability matrix, and test data management.

- **Observations**

	Weak Prompt	ICEPOT Prompt
1. Domain-Specific Coverage	Generates generic cart test cases applicable to any e-commerce application. Important automobile insurance validations (VIN, odometer, contract term, dealer eligibility) are usually missed.	Clearly defines the automobile insurance domain and mandatory business rules, resulting in test cases that cover VIN validation, vehicle information, dealer-product mapping, contract purchase flow, and insurance-specific scenarios.
2. Completeness and Scope Control	The model decides the scope on its own, often providing only UI functional scenarios and missing API, DB, security, and boundary testing.	Explicitly instructs the model to include UI, API, Database, Security, Positive, Negative, Boundary, and Edge Case testing, producing a much more comprehensive test suite.
3. Output Quality and Automation Readiness	Results are usually high-level checklists with inconsistent formatting and little automation value.	Specifies the exact output structure, professional tone, tester persona, and Playwright automation requirements, resulting in audit-ready documentation and automation-friendly test cases

Scenario 3: **API Testing**

- Weak Prompt: "Write API test cases for user registration."
- What is missing in the prompt?
  1. Missing API Contract and Payload Structure
  2. Absence of Specific Business Rules and Constraints
  3. Undefined Target Tooling and Output Format

- Weak Prompt response: (Chatagent: Qwen)

Here is a comprehensive set of API test cases for a standard User Registration endpoint.

Assumptions & Endpoint Details

- **Endpoint:** POST /api/v1/users/register
- **Content-Type:** application/json
- **Mandatory Fields:** username, email, password
- **Optional Fields:** firstName, lastName

---

1. Positive Test Cases (Happy Path)

These test cases verify that the API works correctly when provided with valid data.

Test ID	Scenario	Request Payload	Expected Status	Expected Response / Behavior
TC_POS_01	Register with valid mandatory fields only	{"username": "user1", "email": "user1@test.com", "password": "Pass1234!"}	201 Created	User is created. Response contains userId, username, email, and a success message. Password is **not** returned in plain text.
TC_POS_02	Register with valid mandatory + optional fields	Include firstName and lastName	201 Created	User is created with all provided details saved correctly in the database.
TC_POS_03	Verify database state after successful registration	N/A	N/A	Query the database to ensure the user record exists, password is hashed, and default roles/statuses are applied.

---

2. Negative Test Cases (Data Validation)

These test cases ensure the API properly rejects invalid, missing, or malformed data.

Test ID	Scenario	Request Payload / Action	Expected Status	Expected Response / Behavior
TC_NEG_01	Missing mandatory field (email)	Omit email from JSON	400 Bad Request	Error message: "Email is required."
TC_NEG_02	Missing mandatory field (password)	Omit password from JSON	400 Bad Request	Error message: "Password is required."
TC_NEG_03	Empty request body	{} or Empty	400 Bad Request	Error message indicating

Test ID	Scenario	Request Payload / Action	Expected Status	Expected Response / Behavior
				mandatory fields are missing.
TC_NEG_04	Invalid Email Format	"email": "john.doe@example" (missing .com)	400 Bad Request	Error message: "Invalid email format."
TC_NEG_05	Password too short	"password": "Pass1!" (less than min length)	400 Bad Request	Error message: "Password must be at least 8 characters."
TC_NEG_06	Password lacks complexity	"password": "password123" (no special char/uppercase)	400 Bad Request	Error message detailing password complexity rules.
TC_NEG_07	Username too short / too long	"username": "ab" or 50+ characters	400 Bad Request	Error message: "Username must be between 3 and 30 characters."
TC_NEG_08	Invalid data types	"email": 12345, "password": true	400 Bad Request	Error message: "Invalid data type for field X."
TC_NEG_09	Null values for mandatory fields	"username": null	400 Bad Request	Error message: "Username cannot be null."
TC_NEG_10	Extra/Unknown fields in payload	Add "role": "admin"	201 Created (or 400)	System should ignore unknown fields OR reject them based on strict schema validation. User should **not** be created as an admin.

---

3. Negative Test Cases (Business Logic & State)

These test cases verify rules regarding existing data and system state.

Test ID	Scenario	Request Payload / Action	Expected Status	Expected Response / Behavior
TC_BIZ_01	Register with an already existing Email	Use an email that is already in the DB	409 Conflict	Error message: "Email already registered." (Do not reveal if the *username* is taken to prevent user enumeration, or handle based on specific security policy).
TC_BIZ_02	Register with an already existing Username	Use a username that is already in the DB	409 Conflict	Error message: "Username already taken."
TC_BIZ_03	Case-insensitive duplicate check	Register JohnDoe when johndoe exists	409 Conflict	System should treat usernames/emails as case-insensitive to prevent duplicates.
TC_BIZ_04	Leading/Trailing spaces in Email	"email": " test@test.com "	201 Created	System should automatically trim whitespace before saving and validating.

---

4. Security Test Cases

Crucial for registration endpoints to prevent common web vulnerabilities.

Test ID	Scenario	Request Payload / Action	Expected Status	Expected Response / Behavior
TC_SEC_01	SQL Injection in Username/Email	"username": "admin' OR '1'='1"	400 Bad Request	Input is sanitized. No SQL error is thrown. User is not created.
TC_SEC_02	Cross-Site Scripting (XSS)	"username": "<script>alert(1)</script>"	400 Bad Request	Script tags are stripped or

Test ID	Scenario	Request Payload / Action	Expected Status	Expected Response / Behavior
				rejected. No script is saved to the DB.
TC_SEC_03	Password Leakage in Response	Valid registration payload	201 Created	The response JSON **must not** contain the password field or its hash.
TC_SEC_04	Rate Limiting / Brute Force	Send 100 registration requests in 1 minute from the same IP	429 Too Many Requests	API throttles the requests. Subsequent requests are blocked temporarily.
TC_SEC_05	Mass Assignment / Privilege Escalation	Add "isAdmin": true or "role": "superadmin"	201 Created	The extra fields are ignored. The user is created with standard user privileges only.

---

5. Header & Protocol Test Cases

Testing the HTTP layer and API gateway configurations.

Test ID	Scenario	Request Payload / Action	Expected Status	Expected Response / Behavior
TC_HDR_01	Missing Content-Type Header	Omit Content-Type: application/json	415 Unsupported Media	API rejects the request because it doesn't know how to parse the body.

Test ID	Scenario	Request Payload / Action	Expected Status	Expected Response / Behavior
TC_HDR_02	Wrong Content-Type Header	Content-Type: text/plain	415 Unsupported Media	API rejects the request.
TC_HDR_03	Malformed JSON Body	{"username": "test", "email": "test@test.com" (missing closing brace)	400 Bad Request	API returns a JSON parsing error.
TC_HDR_04	HTTP Method check	Use GET, PUT, DELETE on the register URL	405 Method Not Allowed	Only POST should be allowed for registration.

💡 Best Practices for Automating These Tests

1. **Test Data Cleanup:** Always use a teardown script to delete the users created during positive test cases so they don't cause 409 Conflict errors on subsequent test runs. Alternatively, use dynamically generated emails (e.g., user_{timestamp}@test.com).
2. **Schema Validation:** In your automation framework (like Postman, RestAssured, or PyTest), add JSON Schema assertions to ensure the 201 Created response strictly adheres to the expected format (e.g., userId is a UUID, createdAt is an ISO date).
3. **Environment Independence:** Ensure your tests point to a dedicated QA/Test database, never Production.
4. **Asynchronous Operations:** If registration triggers an email verification link, test that the email was actually sent (using a tool like Mailtrap or checking an AWS SES mock) in a secondary test step.

- ICEPOT Prompt:

[Context] As a API test engineer i should be able to generate test cases for the ecommerce application to test the user registration page.Postman is used for testing.

Instructions :

[Mandatory] The API endpoint is POST /api/v1/register. The required JSON payload includes email (string), password (string), and age (integer).

[Critical] Business rules dictate that passwords must be at least 8 characters with one uppercase letter, the user must be 18 or older,

and duplicate emails must return a 409 Conflict status.

[Mandatory] Generate a comprehensive list of API test cases covering positive, negative, and boundary scenarios for this registration flow.

[Persona]Also Act as a Senior QA Automation Engineer to build a framwork to automating the test framework.

[Output] Provide the test cases in a Markdown table with the columns: Test ID, Scenario, Request Payload, Expected Status Code, and Expected Behavior.

[Tone/Target] Keep the language professional, concise, and technical, targeted at a development team for quick review.

- ICEPOT Prompt Response :

Here are the comprehensive API test cases for the e-commerce user registration endpoint, structured for automation mapping and Postman collection integration.

Test ID	**Scenario**	**Request Payload**	**Expected Status Code**	**Expected Behavior**
TC_REG_01	**Positive:** Valid Registration	{"email":"new@test.com","password":"Pass1234","age":25}	201 Created	User record created. Response includes user ID and success message.
TC_REG_02	**Boundary:** Minimum valid age (18)	{"email":"new@test.com","password":"Pass1234","age":18}	201 Created	User record created successfully.
TC_REG_03	**Boundary:** Minimu	{"email":"new@test.com","password":"Passwor1","age":25}	201 Created	User record created

Test ID	Scenario	Request Payload	Expected Status Code	Expected Behavior
	m valid password (8 chars, 1 uppercase)			successfully.
TC_REG_04	**Negative:** Missing mandatory field (email)	{"password":"Pass1234","age":25}	400 Bad Request	Returns validation error indicating email is required.
TC_REG_05	**Negative:** Missing mandatory field (password)	{"email":"new@test.com","age":25}	400 Bad Request	Returns validation error indicating password is required.
TC_REG_06	**Negative:** Missing mandatory field (age)	{"email":"new@test.com","password":"Pass1234"}	400 Bad Request	Returns validation error indicating age is required.
TC_REG_07	**Negative:** Invalid email format	{"email":"invalid-email","password":"Pass1234","age":25}	400 Bad Request	Returns validation error for invalid email syntax.

Test ID	Scenario	Request Payload	Expected Status Code	Expected Behavior
TC_REG_08	**Boundary:** Password length < 8 characters	{"email":"new@test.com","password":"Pass123","age":25}	400 Bad Request	Returns validation error: password must be at least 8 characters.
TC_REG_09	**Boundary:** Password 8 chars but no uppercase	{"email":"new@test.com","password":"password1","age":25}	400 Bad Request	Returns validation error: password must contain at least one uppercase letter.
TC_REG_10	**Boundary:** Age below minimum (17)	{"email":"new@test.com","password":"Pass1234","age":17}	400 Bad Request	Returns validation error: user must be 18 or older.
TC_REG_11	**Negative:** Duplicate email registration	{"email":"existing@test.com","password":"Pass1234","age":25}	409 Conflict	Returns conflict error: email is already registered.
TC_REG_12	**Negative:** Invalid	{"email":"new@test.com","password":"Pass1234","age":"25"}	400 Bad	Returns schema validatio

Test ID	Scenario	Request Payload	Expected Status Code	Expected Behavior
	data type (age as string)		Request	n error: age must be an integer.

Note for Automation Framework: When implementing in Postman, utilize Postman Data Drivers (CSV/JSON) or the pm.test() scripting layer to iterate through these payloads dynamically, ensuring environment variables are used for the existing@test.com payload in TC_REG_11 to maintain test state independence.

- **Observations**

1. It stopped the guessing game: Since you gave it the exact fields and rules, the AI didn't have to make things up. The test cases actually matched your specific app instead of being generic.
2. You got exactly the format you wanted: Instead of a messy wall of text, it gave you the clean table you asked for, making it ready to copy and use right away.
3. It cut out the fluff: It skipped all the irrelevant filler (like random security tests) and kept the tone professional, just like you'd actually want to share with your dev team.